

Jobsheet 14 Tugas

Praktikum Algoritma dan Struktur Data

Nama: Dafa Naufal Rabbani
Kelas/No.Absen: TI-1G/05
NIM: 254107020086

1. Jawaban:

- Kode Program BinaryTree05:
package Pertemuan14;

```
public class BinaryTree05 {
    Node05 root;

    public BinaryTree05() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa05 mahasiswa) {
        Node05 newNode = new Node05(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node05 current = root;
            while (true) {
                Node05 parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {
                        parent.right = newNode;
                    }
                }
            }
        }
    }
}
```

```

        return;
    }
}
}
}
}

```

```

public void addRekursif(Mahasiswa05 mahasiswa) {
    root = addRekursifProses(root, mahasiswa);
}

```

```

private Node05 addRekursifProses(Node05 current, Mahasiswa05
mahasiswa) {
    if (current == null) {
        return new Node05(mahasiswa);
    }
    if (mahasiswa.ipk < current.mahasiswa.ipk) {
        current.left = addRekursifProses(current.left, mahasiswa);
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {
        current.right = addRekursifProses(current.right, mahasiswa);
    }
    return current;
}

```

```

public void cariMinIPK() {
    if (isEmpty()) {
        System.out.println("Tree kosong");
        return;
    }
    Node05 current = root;
    while (current.left != null) {
        current = current.left;
    }
    System.out.print("Mahasiswa IPK Terkecil -> ");
    current.mahasiswa.tampilInformasi();
}

```

```

public void cariMaxIPK() {
    if (isEmpty()) {
        System.out.println("Tree kosong");
        return;
    }
    Node05 current = root;

```

```

        while (current.right != null) {
            current = current.right;
        }
        System.out.print("Mahasiswa IPK Terbesar -> ");
        current.mahasiswa.tampilInformasi();
    }

    public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
        System.out.println("Daftar Mahasiswa dengan IPK di atas " +
            ipkBatas + ":");
        tampilIPKdiAtasProses(root, ipkBatas);
    }

    private void tampilIPKdiAtasProses(Node05 node, double ipkBatas) {
        if (node != null) {
            tampilIPKdiAtasProses(node.left, ipkBatas);
            if (node.mahasiswa.ipk > ipkBatas) {
                node.mahasiswa.tampilInformasi();
            }
            tampilIPKdiAtasProses(node.right, ipkBatas);
        }
    }

    public boolean find(double ipk) {
        boolean result = false;
        Node05 current = root;
        while (current != null) {
            if (current.mahasiswa.ipk == ipk) {
                result = true;
                break;
            } else if (ipk > current.mahasiswa.ipk) {
                current = current.right;
            } else {
                current = current.left;
            }
        }
        return result;
    }

    public void traversePreOrder(Node05 node) {
        if (node != null) {
            node.mahasiswa.tampilInformasi();
            traversePreOrder(node.left);
        }
    }

```

```

        traversePreOrder(node.right);
    }
}

public void traverseInOrder(Node05 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

public void traversePostOrder(Node05 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.mahasiswa.tampilInformasi();
    }
}

public Node05 getSuccessor(Node05 del) {
    Node05 successor = del.right;
    Node05 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

public void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node05 parent = root;
    Node05 current = root;
    boolean isLeftChild = false;
    while (current != null) {

```

```

if (current.mahasiswa.ipk == ipk) {
    break;
} else if (ipk < current.mahasiswa.ipk) {
    parent = current;
    current = current.left;
    isLeftChild = true;
} else if (ipk > current.mahasiswa.ipk) {
    parent = current;
    current = current.right;
    isLeftChild = false;
}
}
if (current == null) {
    System.out.println("Data tidak ditemukan");
    return;
} else {
    if (current.left == null && current.right == null) {
        if (current == root) {
            root = null;
        } else {
            if (isLeftChild) {
                parent.left = null;
            } else {
                parent.right = null;
            }
        }
    } else if (current.left == null) {
        if (current == root) {
            root = current.right;
        } else {
            if (isLeftChild) {
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        }
    } else if (current.right == null) {
        if (current == root) {
            root = current.left;
        } else {
            if (isLeftChild) {
                parent.left = current.left;
            } else {

```

```

        parent.right = current.left;
    }
}
} else {
    Node05 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}
}
}
}
}

```

- Kode Program BinaryTreeArray05:
package Pertemuan14;

```

public class BinaryTreeArray05 {
    Mahasiswa05[] dataMahasiswa;
    int idxLast;

    public BinaryTreeArray05() {
        this.dataMahasiswa = new Mahasiswa05[15];
        this.idxLast = -1;
    }

    public void add(Mahasiswa05 data) {
        if (idxLast + 1 < dataMahasiswa.length) {
            idxLast++;
            dataMahasiswa[idxLast] = data;
        } else {
            System.out.println("Array Tree Penuh!");
        }
    }
}

```

```

void populateData(Mahasiswa05 dataMhs[], int idxLast) {
    this.dataMahasiswa = dataMhs;
    this.idxLast = idxLast;
}

void traverseInOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            traverseInOrder(2 * idxStart + 1);
            dataMahasiswa[idxStart].tampilInformasi();
            traverseInOrder(2 * idxStart + 2);
        }
    }
}

void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
            traversePreOrder(2 * idxStart + 1);
            traversePreOrder(2 * idxStart + 2);
        }
    }
}

```

- Kode Program BinaryTreeMain05:

```
package Pertemuan14;
```

```

public class BinaryTreeMain05 {
    public static void main(String[] args) {
        BinaryTree05 bst = new BinaryTree05();

        bst.addRekursif(new Mahasiswa05("244160121", "Ali", "A",
3.57));
        bst.addRekursif(new Mahasiswa05("244160221", "Badar", "B",
3.85));
        bst.addRekursif(new Mahasiswa05("244160185", "Candra", "C",
3.21));
        bst.addRekursif(new Mahasiswa05("244160220", "Dewi", "B",
3.54));
        bst.addRekursif(new Mahasiswa05("244160131", "Devi", "A",
3.72));
    }
}

```

```

        bst.addRekursif(new Mahasiswa05("244160205", "Ehsan", "D",
3.37));
        bst.addRekursif(new Mahasiswa05("244160170", "Fizi", "B",
3.46));

        System.out.println("\nDaftar semua mahasiswa (InOrder
Traversal:");
        bst.traverseInOrder(bst.root);

        System.out.println("\n=== HASIL TUGAS PRAKTIKUM ===");
        bst.cariMinIPK();
        bst.cariMaxIPK();

        System.out.println();
        bst.tampilMahasiswaIPKdiAtas(3.50);
    }
}

```

- Kode Program BinaryTreeArrayMain05:
package Pertemuan14;

```

public class BinaryTreeArrayMain05 {
    public static void main(String[] args) {
        BinaryTreeArray05 bta = new BinaryTreeArray05();

        Mahasiswa05 mhs1 = new Mahasiswa05("244160121", "Ali", "A",
3.57);
        Mahasiswa05 mhs2 = new Mahasiswa05("244160185", "Candra",
"C", 3.41);
        Mahasiswa05 mhs3 = new Mahasiswa05("244160221", "Badar",
"B", 3.75);
        Mahasiswa05 mhs4 = new Mahasiswa05("244160220", "Dewi",
"B", 3.35);
        Mahasiswa05 mhs5 = new Mahasiswa05("244160131", "Devi",
"A", 3.48);
        Mahasiswa05 mhs6 = new Mahasiswa05("244160205", "Ehsan",
"D", 3.61);
        Mahasiswa05 mhs7 = new Mahasiswa05("244160170", "Fizi", "B",
3.86);

        Mahasiswa05[] dataMahasiswas = {mhs1, mhs2, mhs3, mhs4,
mhs5, mhs6, mhs7, null, null, null};
        int idxLast = 6;
    }
}

```



```

        bta.populateData(dataMahasiswas, idxLast);

        System.out.println("\nInOrder Traversal Mahasiswa: ");
        bta.traverseInOrder(0);

        System.out.println("\n=== HASIL TUGAS PRAKTIKUM ===");
        System.out.println("PreOrder Traversal Mahasiswa: ");
        bta.traversePreOrder(0);
    }
}

```

- Hasil Running BinaryTreeMain05:

```

PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData> & 'C:\Users\Dapaa\AppData\Roaming\Code\User\workspaceStorage\faaac69d095\Pertemuan14.BinaryTreeMain05'

Daftar semua mahasiswa (InOrder Traversal):
NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.37
NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.46
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

=== HASIL TUGAS PRAKTIKUM ===
Mahasiswa IPK Terkecil -> NIM: 244160185 Nama: Candra Kelas: C IPK: 3.21
Mahasiswa IPK Terbesar -> NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85

Daftar Mahasiswa dengan IPK di atas 3.5:
NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.54
NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57
NIM: 244160131 Nama: Devi Kelas: A IPK: 3.72
NIM: 244160221 Nama: Badar Kelas: B IPK: 3.85
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>

```

- Hasil Running BinaryTreeArrayMain:

```
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData> & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -cp 'C:\Users\Dapaa\AppData\Roaming\Code\User\workspaceStorage\1\workspace\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData_4bef7bdf\bin' 'Pertemuan14.BinaryTreeArrayMain05'
```

InOrder Traversal Mahasiswa:

NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35

NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41

NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48

NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75

NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86

=== HASIL TUGAS PRAKTIKUM ===

PreOrder Traversal Mahasiswa:

NIM: 244160121 Nama: Ali Kelas: A IPK: 3.57

NIM: 244160185 Nama: Candra Kelas: C IPK: 3.41

NIM: 244160220 Nama: Dewi Kelas: B IPK: 3.35

NIM: 244160131 Nama: Devi Kelas: A IPK: 3.48

NIM: 244160221 Nama: Badar Kelas: B IPK: 3.75

NIM: 244160205 Nama: Ehsan Kelas: D IPK: 3.61

NIM: 244160170 Nama: Fizi Kelas: B IPK: 3.86

```
PS D:\Polinema\Semester2\Tugas\PASD\PraktikAlgoritmaDanStrukturData>
```